

AI Sales Outreach Automation — Complete Project Overview

A deep-dive reference covering architecture, data flow, LangGraph agents, n8n workflows, and every layer of the stack. Read top-to-bottom for first understanding; use headings to jump when referencing.

Table of Contents

1. [What This Project Does](#)
 2. [Mental Model — Three Planes](#)
 3. [Tech Stack](#)
 4. [Directory Structure](#)
 5. [End-to-End Flow](#)
 6. [Database Layer](#)
 7. [LangGraph Agents — Deep Dive](#)
 - 7.1 [Research Agent](#)
 - 7.2 [Personalization Agent](#)
 - 7.3 [Reply Classifier](#)
 - 7.4 [Follow-up Agent](#)
 - 7.5 [How LangGraph Works](#)
 8. [n8n Workflows — Deep Dive](#)
 - 8.1 [Campaign Launcher](#)
 - 8.2 [Gmail Reply Monitor](#)
 - 8.3 [Follow-up Scheduler](#)
 9. [FastAPI Backend](#)
 10. [Frontend \(Next.js\)](#)
 11. [Current Build Status](#)
-

1. What This Project Does

This is a **B2B cold-email automation platform**. Given a list of company leads (CSV or Google Sheets), it:

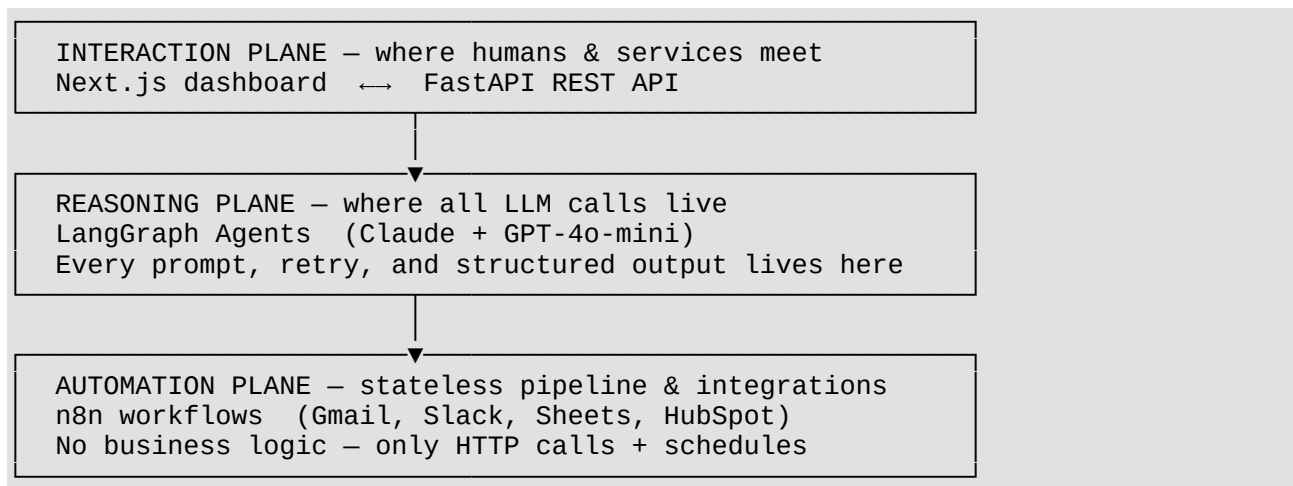
1. **Researches** each company — scrapes their website, pulls recent news, infers their tech stack
2. **Writes a personalized email** — not a template with `{firstName}` swapped in, but a

genuinely specific email that references a concrete fact about that company

3. **Checks compliance** — no spam words, no unverifiable claims, correct length
4. **Sends via Gmail** — the user's own Gmail account, so it doesn't look like bulk mail
5. **Monitors for replies** — every 15 minutes, scans Gmail for responses
6. **Classifies replies** — "interested", "not interested", "wants a meeting", "unsubscribe", etc.
7. **Schedules follow-ups** — at 3 / 7 / 14 days with different strategies: bump, value-add, break-up
8. **Syncs the CRM** — HubSpot/Airtable/Notion

The product differentiator is **personalization quality**. Every email must reference a fact specific to that company found during research.

2. Mental Model — Three Planes



The golden rule: n8n is the *pipeline*, LangGraph is the *brain*. Never put `if/else` decision logic in n8n — it calls FastAPI and acts on the response.

3. Tech Stack

Layer	Technology	Why
Frontend	Next.js 14 (App Router), TypeScript, Tailwind, shadcn/ui	Server components + React Query for async
Backend API	FastAPI + Python 3.11	Async-native, Pydantic v2, excellent OpenAPI
Agents	LangGraph + LangChain	Stateful DAG with conditional edges; easy to add retry loops
LLMs	Claude Sonnet (synthesis/writing) + GPT-4o-	Best-of-both: Claude for nuanced writing, GPT-4o-mini

Layer	Technology	Why
	mini (quality checks)	for fast structured checks
Automation	n8n (self-hosted Docker)	Visual workflow editor, native Gmail + Sheets + Slack nodes
Relational DB	PostgreSQL 15	Source of truth for all entities
Vector DB	Qdrant	Semantic search over past research + best-performing email templates
Cache / Queue	Redis	Rate limits (100 Gmail/day), short-lived locks
Deploy	Docker Compose	Single make dev boots everything

4. Directory Structure

```
/
├── frontend/           Next.js app (UI)
│   └── src/app/        App Router pages
├── backend/           FastAPI app
│   ├── api/           Route handlers (campaigns, leads, emails, auth,
│   │                 webhooks, internal)
│   ├── core/          Config, DB session, vector store client, embeddings,
│   │                 exceptions
│   ├── models/         SQLAlchemy ORM models
│   ├── schemas/        Pydantic request/response schemas
│   └── agents/         LangGraph agents (importable package, isolated from
│   │                 backend)
│       ├── research_agent.py
│       ├── personalization_agent.py
│       ├── reply_classifier.py
│       ├── followup_agent.py
│       └── prompts/     All system prompts live here – diff-reviewable
│                       separately
├── n8n-workflows/     Exported n8n JSON – import via UI
├── infra/             Docker Compose, nginx config
└── docs/              Specs and architecture diagrams
```

5. End-to-End Flow

Phase A — Campaign Launch

```
User (browser)
  |
  └─ POST /api/campaigns { name, settings }
      ▼
FastAPI
  |
  └─ INSERT campaigns row, status='draft'

      [User uploads CSV]
      POST /api/leads/upload
      → parse CSV, validate emails, bulk INSERT leads (status='new')
```

```

    [User clicks "Launch"]
    PATCH /api/campaigns/{id}/status { "status": "active" }
    → POST to n8n webhook { "campaign_id": "uuid" }
  ▼
n8n: campaign_launcher workflow

  GET /api/internal/leads?campaign_id=...&status=new
  → receives list of leads

  For each lead (batch size 1, 30-second wait between):
    POST /api/internal/trigger-research { lead_id }
    POST /api/internal/trigger-personalization { lead_id }
    Gmail Node: send email
    PATCH /api/internal/leads/{id} { status: "email_sent",
gmail_message_id }
    [on error] → Slack notify, continue to next lead
  ▼
FastAPI /internal/trigger-research
  → calls run_research_agent(lead) [LangGraph]
  → stores result in leads.research_data (JSONB)
  → upserts Qdrant vector: company_research collection
  ▼
FastAPI /internal/trigger-personalization
  → calls run_personalization_agent(lead, research, campaign_context)
[LangGraph]
  → INSERT emails row
  → returns { subject, body, full_email } to n8n
  ▼
n8n Gmail Node sends email → stores gmail_message_id back via PATCH

```

Phase B — Reply Detection (every 15 minutes)

```

n8n: gmail_reply_monitor (cron, every 15 min)

  GET /api/internal/sent-emails/recent (last 14 days)
  Gmail Search: is:inbox, threaded on those message IDs

  For each new reply found:
    POST /api/webhooks/n8n/reply-received
      { gmail_message_id, reply_text, received_at }
    Slack: notify #sales-replies
    Google Sheets: log row
  ▼
FastAPI webhook handler
  → INSERT replies row
  → calls run_reply_classifier(reply_text) [single LLM call]
  → UPDATE leads.status based on intent
  → UPDATE crm_updates for HubSpot sync

```

Phase C — Follow-ups (daily 09:00)

```

n8n: followup_scheduler (cron, 09:00 daily)

  GET /api/internal/leads-needing-followup
  → returns leads with no reply at 3/7/14 day marks

  For each lead:
    POST /api/internal/trigger-followup { lead_id }
    IF response.should_send == true:
      Gmail Node: send follow-up
      PATCH /api/internal/leads/{id} { status: "email_sent" }

```

```
ELSE: log only
Google Sheets: daily summary row
```

6. Database Layer

PostgreSQL Schema

```
campaigns
├── id (UUID PK)
├── name (TEXT)
├── status (ENUM: draft / active / paused / completed)
└── settings (JSONB) ← tone, target_audience, product, value_prop, case_study

leads
├── id (UUID PK)
├── campaign_id (FK → campaigns)
├── company_name, website, contact_name, email
├── status (ENUM: new / researched / email_sent / replied / meeting_booked /
unsubscribed)
└── research_data (JSONB) ← Research Agent output stored here

emails
├── id (UUID PK)
├── lead_id (FK → leads)
├── subject, body
├── type (ENUM: outreach / followup)
├── sent_at, opened_at, replied_at
└── gmail_message_id ← used for reply threading

replies
├── id (UUID PK)
├── email_id (FK → emails)
├── content (TEXT)
├── classified_as (ENUM: interested / not_interested / meeting_request /
unsubscribe / unknown)
└── received_at

oauth_tokens
├── provider (ENUM: gmail / hubspot / slack)
├── access_token_enc (BYTEA, Fernet-encrypted)
└── refresh_token_enc (BYTEA, Fernet-encrypted)
```

Qdrant (Vector Memory)

Two collections, both using OpenAI text-embedding-3-small (1536 dims, Cosine distance):

company_research — stores research summaries so similar companies can be found

```
# Upserted after each Research Agent run
await vector_store.upsert_company_research(
    lead_id="uuid",
    summary_text="ABC Corp is a logistics SaaS...",
    metadata={"company_name": "ABC Corp", "website": "abc.com"}
)
```

email_templates — stores high-performing email bodies tagged with industry + pain_point

```
# Queried during Personalization Agent
templates = await vector_store.get_best_templates(
    industry="Logistics SaaS",
    pain_point="manual outbound prospecting"
)
# Returns top-N templates sorted by reply_rate
```

7. LangGraph Agents — Deep Dive

7.5 How LangGraph Works (the pattern)

Before diving into each agent, here's the pattern every agent follows:

```
# 1. Define state as a TypedDict – this is the "memory" of the graph
class MyState(TypedDict):
    input: str
    intermediate: str | None
    output: dict | None
    retry_count: int

# 2. Define node functions – each takes state, returns partial update
async def node_a(state: MyState) -> dict:
    result = await call_llm(state["input"])
    return {"intermediate": result}  # only return what changed

async def node_b(state: MyState) -> dict:
    validated = validate(state["intermediate"])
    return {"output": validated}

# 3. Define routing function for conditional edges
def route(state: MyState) -> str:
    if state["output"] is None and state["retry_count"] < 2:
        return "retry"
    return "end"

# 4. Build and compile the graph
graph = StateGraph(MyState)
graph.add_node("node_a", node_a)
graph.add_node("node_b", node_b)
graph.add_edge(START, "node_a")
graph.add_edge("node_a", "node_b")
graph.add_conditional_edges("node_b", route, {"retry": "node_a", "end": END})
compiled = graph.compile()

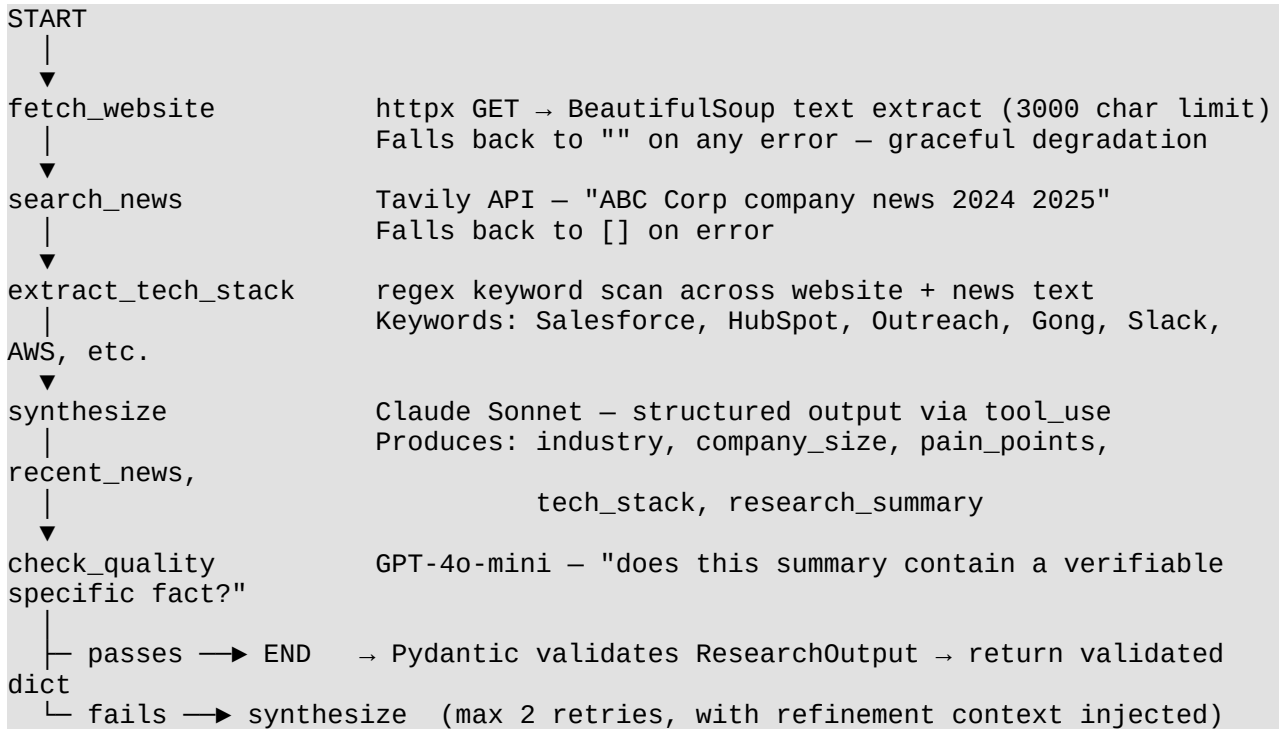
# 5. Run it – state flows through nodes, LangGraph manages the DAG
final_state = await compiled.ainvoke({"input": "...", "retry_count": 0, ...})
```

The key insight: **each node only sees the full state and returns the fields it changed**. LangGraph merges the partial update into the full state before calling the next node.

7.1 Research Agent

File: [agents/research_agent.py](#) Entry point: `run_research_agent(lead: dict) -> dict`

Graph



State

```
class ResearchState(TypedDict):
    lead: dict # {"company_name": str, "website": str}
    raw_website_text: str | None # "" on fetch failure
    news_results: list[dict] # [] on Tavily failure
    tech_stack_hints: list[str] # keyword matches
    synthesized: dict | None # raw Claude tool_use output
    quality_ok: bool # set by check_quality node
    refine_count: int # 0, 1, or 2
```

How Claude is called (tool_use pattern)

```
RESEARCH_OUTPUT_TOOL = {
    "name": "submit_research",
    "description": "Submit the structured research findings.",
    "input_schema": {
        "type": "object",
        "properties": {
            "industry": {"type": "string"},
            "company_size": {"type": "string"},
            "pain_points": {"type": "array", "items": {"type": "string"}},
            "recent_news": {"type": "array", "items": {"type": "string"}},
            "tech_stack": {"type": "array", "items": {"type": "string"}},
            "research_summary": {"type": "string"}},
        "required": ["industry", "company_size", "pain_points", "recent_news", "tech_stack", "research_summary"],
    },
}

resp = await client.messages.create(
    model="claude-sonnet-4-20250514",
    system=SYNTHESIS_SYSTEM,
```

```

tools=[RESEARCH_OUTPUT_TOOL],
tool_choice={"type": "tool", "name": "submit_research"}, # forces the tool
call
messages=[{"role": "user", "content": user_message}],
)
tool_use = next(b for b in resp.content if b.type == "tool_use")
return tool_use.input # always a valid dict matching the schema

```

Why tool_choice: force? It guarantees Claude returns structured JSON rather than prose.
No regex parsing needed.

Quality check + retry loop

```

def _route_after_quality(state: ResearchState) -> str:
    if state["quality_ok"] or state["refine_count"] >= 2:
        return "end" # pass or max retries hit
    return "synthesize" # loop back with refinement context

# On re-entry into synthesize, refine_context is injected into the prompt:
# "Previous attempt failed. Write a more specific summary that references
# a concrete fact from the source material. Prior summary: <prior>"

```

Output

```

{
  "industry": "Logistics SaaS",
  "company_size": "50-200",
  "pain_points": ["manual outbound prospecting", "outbound scaling"],
  "recent_news": ["Expanded to Europe Q1 2025"],
  "tech_stack": ["Salesforce", "Outreach.io"],
  "research_summary": "ABC Logistics is a mid-size logistics SaaS firm that
expanded into Europe in Q1 2025; recent sales hires suggest their outbound
motion is manual."
}

```

7.2 Personalization Agent (with embedded Compliance)

File: [agents/personalization_agent.py](#) **Entry point:** `run_personalization_agent(lead, research, campaign_context) -> dict`

Graph

```

START
  |
  ▼
retrieve_templates      Qdrant: get_best_templates(industry, pain_point)
                        Returns top-N email bodies ranked by reply_rate
                        Falls back to [] on error – agent continues without
  |
templates
  |
  ▼
draft_email            Claude Sonnet – few-shot from retrieved templates
                        Produces: subject, opening_line, body, cta, full_email
  |
  ▼
compliance_check       Two-stage:
                        (a) Deterministic: spam words, body > 200 words, subject
                        > 60 chars
                        (b) Semantic (GPT-4o-mini): unverifiable claims,
                        opening_line not grounded in research
  |
  > 60 chars
  |
opening_line not grounded in research

```



```

└─ clean → END      → return draft
└─ violations → refine → compliance_check    (max 2 iterations)

```

State

```

class PersonalizationState(TypedDict):
    lead: dict
    research: dict
    campaign_context: dict    # tone, product, value_prop, case_study
    templates: list[dict]    # from Qdrant
    draft: dict | None        # current email draft
    compliance_violations: list[str]
    refine_count: int

```

Compliance check — two stages

```

async def compliance_check(state: PersonalizationState) -> dict:
    draft = state["draft"]
    violations = []

    # Stage 1: deterministic – no LLM cost
    body_lower = draft["body"].lower()
    SPAM_WORDS = {"guaranteed", "free money", "act now", "limited time",
                  "click here", "100%", "unlimited", "risk-free"}
    for word in SPAM_WORDS:
        if word in body_lower:
            violations.append(f"spam trigger word: '{word}'")

    if len(draft["body"].split()) > 200:
        violations.append("body too long (over 200 words)")

    if len(draft["subject"]) > 60:
        violations.append(f"subject too long ({len(draft['subject'])} chars)")

    if violations:
        return {"compliance_violations": violations} # short-circuit, skip LLM

    # Stage 2: semantic – GPT-4o-mini checks claims and opening line
    result = await _call_openai_compliance(draft, research_summary)
    return {"compliance_violations": result.get("violations", [])}

```

Refinement — violations fed back to Claude

```

async def refine(state: PersonalizationState) -> dict:
    # _call_claude_draft with violations list → uses REFINE_SYSTEM prompt
    # The violations are appended to the user message:
    # "VIOLATIONS TO FIX IN THIS REVISION:
    # - spam trigger word: 'guaranteed'
    # - subject too long (72 chars)"
    result = await _call_claude_draft(
        state["lead"], state["research"], state["campaign_context"],
        state["templates"],
        violations=state["compliance_violations"], # ← key difference
    )
    return {"draft": result, "refine_count": state["refine_count"] + 1}

```

Output

```

{
  "subject": "Question about your Europe expansion",

```

```
"opening_line": "Saw your expansion into Europe – that usually 3x's outbound ops complexity.",
"body": "Hi Sarah,\n\nNoticed ABC Logistics expanded into Europe in Q1...",
"cta": "Open to a 15-min chat next Tuesday?",
"full_email": "<full assembled email>"
}
```

7.3 Reply Classifier

File: [agents/reply_classifier.py](#) **Entry point:** `run_reply_classifier(reply_text, prior_email=None)` -> `ClassificationResult`

This is the simplest agent — a **single GPT-4o-mini call** with structured output. No graph needed.

```
# The intent → action mapping is enforced server-side, NOT trusted from the LLM
INTENT_TO_ACTION = {
    "interested": "schedule_call",
    "not_interested": "close_lead",
    "meeting_request": "schedule_call",
    "unsubscribe": "unsubscribe_lead",
    "needs_more_info": "reply_with_info",
    "unknown": "wait",
}

class ClassificationResult(BaseModel):
    intent: Intent
    confidence: float = Field(ge=0.0, le=1.0)
    suggested_next_action: NextAction
    key_phrases: list[str]

    @model_validator(mode="after")
    def _enforce_action_mapping(self) -> "ClassificationResult":
        # Even if LLM returns wrong action, we override it deterministically
        canonical = INTENT_TO_ACTION[self.intent]
        if self.suggested_next_action != canonical:
            self.suggested_next_action = canonical
        return self
```

Why override the LLM's suggested action? Intent classification is hard and the LLM can be right about the intent but wrong about the action. The mapping is business logic — it belongs in code, not in a prompt.

Output

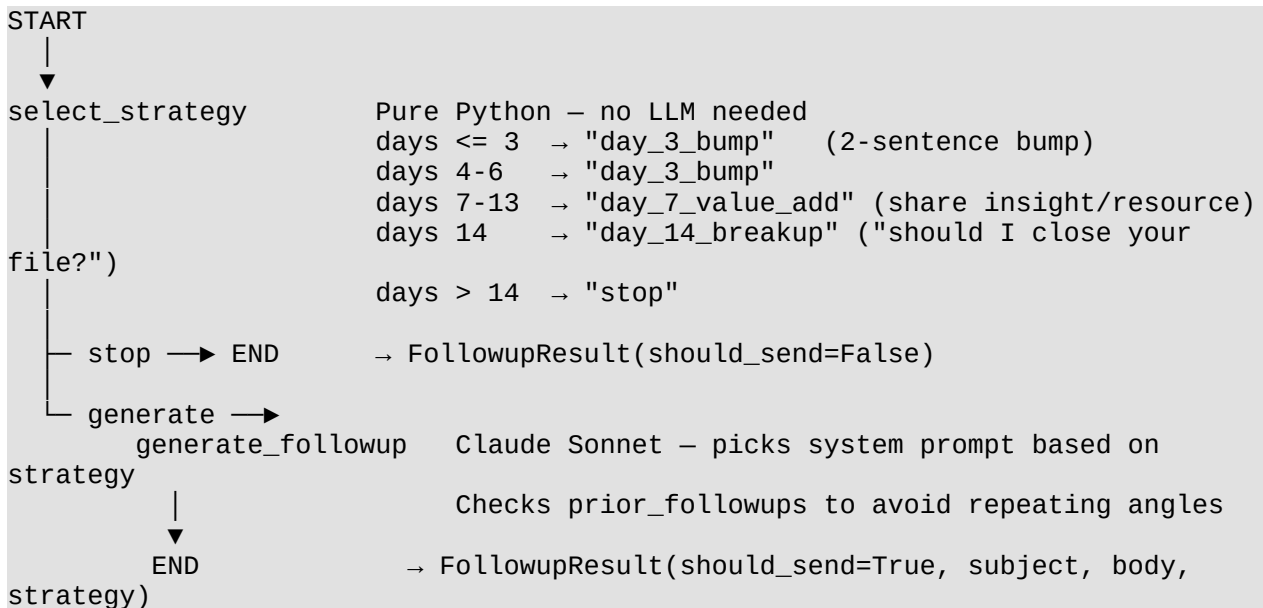
```
{
  "intent": "interested",
  "confidence": 0.86,
  "suggested_next_action": "schedule_call",
  "key_phrases": ["sounds interesting", "next week works"]
}
```

7.4 Follow-up Agent

File: [agents/followup_agent.py](#) **Entry point:** `run_followup_agent(lead_id, days_since_last_touch, original_email, prior_followups, research)`

-> FollowupResult

Graph



Strategy selection (pure Python, no LLM)

```
def select_strategy(state: FollowupState) -> dict:
    days = state["days_since_last_touch"]
    if days > 14:
        return {"strategy": "stop", "should_send": False}
    elif days >= 14:
        strategy = "day_14_breakup"
    elif days >= 7:
        strategy = "day_7_value_add"
    else:
        strategy = "day_3_bump"
    return {"strategy": strategy, "should_send": True}
```

Angle deduplication

```
async def generate_followup(state: FollowupState) -> dict:
    prior_strategies = [f.get("type") for f in state["prior_followups"]]
    if strategy in prior_strategies:
        system_prompt += "\nAdjust the angle slightly – this strategy was already used."
    # ...
```

Output

```
{
  "should_send": true,
  "subject": "Quick bump on my last note",
  "body": "Hi Sarah – just bumping this up in case it got buried...",
  "strategy": "day_3_bump"
}
```

8. n8n Workflows — Deep Dive

n8n runs at `http://localhost:5678` (Docker). All workflows are version-controlled as JSON in `/n8n-workflows/`. Import via the n8n UI.

Contract with FastAPI: n8n only calls `/api/internal/*` endpoints with `X-Internal-Token` header. It never has DB access. It never transforms data beyond field mapping.

8.1 Campaign Launcher

File: [n8n-workflows/campaign_launcher.json](#) **Trigger:** Webhook POST — fired by FastAPI when campaign status → active

```
Webhook (POST)
  payload: { "campaign_id": "uuid" }
  |
  ▼ Node 1
HTTP GET /api/internal/leads?campaign_id={{ $json.campaign_id }}&status=new
  → receives array of lead objects
  |
  ▼ Node 2
Split In Batches (size: 1)
  → processes one lead at a time
  |
  ▼ Node 3
HTTP POST /api/internal/trigger-research
  body: { "lead_id": "{{ $json.id }}" }
  → FastAPI runs Research Agent, stores result in DB
  → returns: { "data": { "research_data": {...} } }
  |
  ▼ Node 4
HTTP POST /api/internal/trigger-personalization
  body: { "lead_id": "{{ $json.id }}" }
  → FastAPI runs Personalization Agent
  → returns: { "data": { "subject": "...", "body": "...", "full_email":
"... " } }
  |
  ▼ Node 5
Gmail: Send Email
  to: {{ $('Split In Batches').item.json.email }}
  subject: {{ $json.data.subject }}
  body: {{ $json.data.full_email }}
  → returns gmail_message_id
  |
  ▼ Node 6
HTTP PATCH /api/internal/leads/{{ lead_id }}
  body: { "status": "email_sent", "gmail_message_id": "{{ $json.id }}" }
  |
  ▼ Node 7
Wait: 30 seconds
  → rate-limit between leads to avoid Gmail spam flags
  |
  ▼ (loops back to Node 2 for next lead)

On any non-2xx at any node:
  → Error branch → Slack: notify #sales-errors with full payload
  → Continue to next lead (don't abort campaign)
```

Idempotency: Every HTTP node sends `Idempotency-Key: {{ $execution.id }}-{{ $json.id }}` so retried executions don't double-send.

8.2 Gmail Reply Monitor

File: [n8n-workflows/gmail_reply_monitor.json](#) **Trigger:** Schedule — every 15 minutes

```
Cron (every 15 min)
├─ ▼ Node 1
HTTP GET /api/internal/sent-emails/recent
  → returns list of { gmail_message_id, lead_id, email_id } for emails sent in last 14 days
├─ ▼ Node 2
Gmail: Search Messages
  query: "is:inbox"
  → threads on the gmail_message_ids from Node 1
  → finds only replies not previously processed
├─ ▼ Node 3
For Each Reply:
├─ Node 3a
  HTTP POST /api/webhooks/n8n/reply-received
  body: {
    "gmail_message_id": "{{ $json.threadId }}",
    "reply_text": "{{ $json.snippet }}",
    "received_at": "{{ $json.internalDate }}"
  }
  → FastAPI: INSERT reply, run Reply Classifier, UPDATE lead status
├─ Node 3b
  Slack: Send Message to #sales-replies
  text: "Reply from {{ company_name }}: {{ reply_snippet }}"
└─ Node 3c
  Google Sheets: Append Row to "Reply Activity" sheet
  columns: [timestamp, company_name, email, reply_snippet, classified_as]
```

8.3 Follow-up Scheduler

File: [n8n-workflows/followup_scheduler.json](#) **Trigger:** Schedule — daily at 09:00 server time

```
Cron (09:00 daily)
├─ ▼ Node 1
HTTP GET /api/internal/leads-needing-followup
  → FastAPI queries: leads with status='email_sent', no reply, WHERE last_touch was exactly 3, 7, or 14 days ago
  → returns array of lead objects with days_since_last_touch
├─ ▼ Node 2
For Each Lead:
├─ ▼ Node 3
HTTP POST /api/internal/trigger-followup
  body: { "lead_id": "{{ $json.id }}" }
  → FastAPI runs Follow-up Agent
  → returns: { "data": { "should_send": true/false, "subject": "...", "body": "...", "body_html": "" } }
├─ ▼ Node 4
IF {{ $json.data.should_send }} == true:
├─ TRUE branch:
```

```

    Gmail: Send Email
      subject: {{ $json.data.subject }}
      body: {{ $json.data.body }}

    HTTP PATCH /api/internal/leads/{{ lead_id }}
      body: { "status": "email_sent" }

  └─ FALSE branch:
      Log only – lead is past the follow-up window

  ▼ Node 5 (after all leads processed)
Google Sheets: Append Row to "Daily Follow-up Summary"
  columns: [date, leads_processed, emails_sent, skipped]

```

9. FastAPI Backend

API Design

All responses use the envelope: { "data": ..., "error": null, "meta": { ... } }

Public Routes (called by Next.js frontend)

Method	Path	What it does
POST	/api/campaigns	Create campaign
GET	/api/campaigns	List campaigns (paginated)
GET	/api/campaigns/{id}	Campaign detail + stats
PATCH	/api/campaigns/{id}/status	Launch/pause — triggers n8n on → active
POST	/api/leads/upload	CSV upload → bulk insert
GET	/api/leads	List with filters (campaign_id, status)
GET	/api/leads/{id}	Lead detail + email history
GET	/api/emails	Email list for a lead
GET	/api/auth/gmail	Get Gmail OAuth URL
GET	/api/auth/gmail/callback	Exchange OAuth code, store encrypted tokens

Internal Routes (called by n8n only, X-Internal-Token required)

Method	Path	What it does
POST	/api/internal/trigger-research	Run Research Agent for one lead
POST	/api/internal/trigger-	Run Personalization Agent, persist email

Method	Path	What it does
	personalization	
GET	/api/internal/leads-needing-followup	Leads at 3/7/14-day follow-up windows
POST	/api/internal/trigger-followup	Run Follow-up Agent for one lead
GET	/api/internal/sent-emails/recent	Emails sent in last 14 days (for Gmail scan)

Webhook Routes (called by n8n after external events)

Method	Path	What it does
POST	/api/webhooks/n8n/reply-received	Reply detected → classify → update lead
POST	/api/webhooks/n8n/email-opened	Tracking pixel fired → store opened_at

Error codes

Code	HTTP	Meaning
validation_failed	422	Bad request body
not_found	404	Resource not found
gmail_quota_exceeded	429	100/day Gmail limit hit (tracked in Redis)
agent_output_invalid	422	LangGraph agent failed Pydantic validation
upstream_error	502	Tavily / Firecrawl / LLM API down

10. Frontend (Next.js)

App Router structure. TanStack Query for all server state. No Redux/Zustand.

Key pages (Phase 5, not yet built):

- /campaigns — list + create
- /campaigns/[id] — detail: leads table, email preview, stats
- /campaigns/[id]/leads/[leadId] — lead detail: research data, email thread, reply classification
- /settings — Gmail OAuth connect button, account status

11. Current Build Status

Phase	What	Status
Phase 0	Scaffold (Docker, project structure)	Complete
Phase 1	Data layer (SQLAlchemy models, Alembic migrations, schemas)	Complete
Phase 2	FastAPI backend (all routes, Gmail OAuth, GmailService)	Complete
Phase 3	LangGraph Agents (Research, Personalization, Reply Classifier, Follow-up)	Complete — 81 tests passing
Phase 4	n8n Workflows (campaign_launcher, reply_monitor, followup_scheduler)	☐ Not started
Phase 5	Frontend (Next.js pages)	☐ Not started
Phase 6	Integration tests	☐ Not started
Phase 7	Docker & deployment	☐ Not started

Key Design Decisions

- 1. Why tool_use / structured output instead of parsing LLM prose?** The Anthropic `tool_choice: force` pattern guarantees JSON output matching a schema. No regex, no `json.loads` on freeform text, no silent schema drift. GPT-4o-mini uses `response_format: json_object` for the same reason.
- 2. Why two LLMs (Claude + GPT-4o-mini)?** Claude Sonnet writes better B2B prose — more specific, less generic. GPT-4o-mini is faster and cheaper for structured binary checks (quality gate: yes/no, compliance: violations list). Using the right tool for each sub-task.
- 3. Why LangGraph instead of a plain async pipeline?** The retry loops (quality → re-synthesize, compliance → refine) need conditional edges. LangGraph handles the DAG + state accumulation cleanly. Without it, you'd write manual recursion with shared mutable state.
- 4. Why n8n instead of Celery or cron?** n8n has native Gmail + Sheets + Slack nodes with OAuth built in — that's months of integration work for free. The trade-off is that business logic can't live in n8n (it's a black box). The rule is strict: n8n calls FastAPI, FastAPI decides.
- 5. Why Qdrant alongside Postgres?** Semantic retrieval ("find email templates for logistics companies with manual-ops pain points") doesn't map to SQL. Qdrant handles nearest-neighbor on embeddings. Postgres handles relational queries. Neither tries to do the other's job.

